Capstone 6 – Programs and Explanations

Project question: Can AI explain beginner Python code clearly and correctly, and where does it mislead learners?

Ten short programs (loops, functions, lists, dictionaries, file handling, strings). Each was run for real; the output shown is the actual output. Each has line-by-line explanation, an output trace and a practice problem.

P01 – Loop: running total (loops)

```
total = 0
for n in range(1, 6):
    total += n
    print("n =", n, "total =", total)
print("Final total:", total)
```

- Line 1: create `total` and start it at 0.
- Line 2: `range(1, 6)` gives 1, 2, 3, 4, 5. The end value 6 is **excluded**.
- Line 3: add `n` to `total`.
- Line 4: print n and the running total each time.
- Line 5: after the loop ends, print the final total.

Trace: n=1 → 1, n=2 → 3, n=3 → 6, n=4 → 10, n=5 → 15.

Actual output: `n = 1 total = 1` … `n = 5 total = 15`, then `Final total: 15`.

Practice: Print the sum of the even numbers from 2 to 20.

P02 – While loop: countdown

```
count = 3
while count > 0:
    print(count)
    count -= 1
print("Liftoff!")
```

- `while count > 0` repeats as long as the condition is True.
- `count -= 1` reduces the counter; without it the loop never ends.

Trace: 3 → print, becomes 2 → print, becomes 1 → print, becomes 0 → stop.

Actual output: `3`, `2`, `1`, `Liftoff!`

Practice: Count down from 10 in steps of 2.

P03 – Function: factorial

```
def factorial(n):
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

print(factorial(5))
print(factorial(0))
```

- `def` defines a function with parameter `n`.
- The loop multiplies 2 × 3 × … × n. For n = 0 or 1 the range is empty, so the result stays 1.
- `return` sends the value back to the caller.

Trace for 5: 1×2=2, ×3=6, ×4=24, ×5=120.

Actual output: `120` then `1`.

Practice: Write `power(base, exp)` without using `**`.

P04 – Lists: add, insert, slice, remove

```
fruits = ["apple", "banana", "cherry"]
fruits.append("date")
fruits.insert(1, "blueberry")
print(fruits)
print(fruits[1:3])
print(fruits[-1])
fruits.remove("banana")
print(len(fruits), fruits)
```

- `append` adds at the end; `insert(1, x)` puts x at index 1 and shifts the rest right.
- `fruits[1:3]` takes indexes 1 and 2 (end index excluded).
- `fruits[-1]` is the last item.
- `remove` deletes the first matching value.

Actual output:

```
['apple', 'blueberry', 'banana', 'cherry', 'date']
['blueberry', 'banana']
date
4 ['apple', 'blueberry', 'cherry', 'date']
```

Practice: Reverse the list without using `.reverse()`.

P05 – List comprehension

```
nums = [1, 2, 3, 4, 5, 6]
evens = [n for n in nums if n % 2 == 0]
squares = [n ** 2 for n in evens]
print(evens)
print(squares)
```

- `n % 2 == 0` is True when n is even; the comprehension keeps only those.
- `n ** 2` means n squared. Squares are made from `evens`, not from `nums`.

Actual output: `[2, 4, 6]` and `[4, 16, 36]`.

Practice: Build a list of cubes of the odd numbers up to 9.

P06 – Dictionary: word count

```
text = "the cat and the hat"
counts = {}
for word in text.split():
    counts[word] = counts.get(word, 0) + 1
print(counts)
```

- `split()` breaks the text at spaces into a list of words.
- `counts.get(word, 0)` returns the current count, or 0 if the word is new.
- Adding 1 and storing it updates the dictionary.

Actual output: `{'the': 2, 'cat': 1, 'and': 1, 'hat': 1}`

Practice: Make it ignore upper/lower case.

P07 – File handling: write then read

```
with open("notes.txt", "w") as f:
    f.write("Line one\nLine two\nLine three\n")

with open("notes.txt") as f:
    for number, line in enumerate(f, start=1):
        print(number, line.strip())
```

- `"w"` creates or **overwrites** the file. `with` closes it automatically.
- `\n` is a new line. `enumerate(f, start=1)` numbers lines from 1.
- `strip()` removes the trailing newline so no blank lines appear.

Actual output: `1 Line one`, `2 Line two`, `3 Line three`

Practice: Print only lines containing the word "two".

P08 – File handling: append and count

```
with open("log.txt", "w") as f:
    f.write("a\n")
with open("log.txt", "a") as f:
    f.write("b\n")
with open("log.txt") as f:
    lines = f.readlines()
print(len(lines), lines)
```

- `"w"` writes `a`; `"a"` **adds** `b` to the end without erasing.
- `readlines()` returns a list of lines, each still ending in `\n`.

Actual output: `2 ['a\n', 'b\n']`

Practice: Count the words in a file.

P09 – Function default argument (a known trap)

```
def add_item(item, basket=[]):
    basket.append(item)
    return basket

print(add_item("apple"))
print(add_item("pear"))
```

- The default list `[]` is created **once**, when the function is defined, and shared by every call that omits `basket`.
- So the second call adds to the same list.
- Fix: use `basket=None` and create `[]` inside the function.

Actual output: `['apple']` then `['apple', 'pear']`.

Practice: Rewrite `add_item` so each call starts with a fresh list.

P10 – Strings: palindrome

```
def is_palindrome(text):
    cleaned = "".join(ch.lower() for ch in text if ch.isalnum())
    return cleaned == cleaned[::-1]

print(is_palindrome("Racecar"))
print(is_palindrome("A man, a plan, a canal: Panama"))
print(is_palindrome("hello"))
```

- The generator keeps only letters/digits (`isalnum`) and lowercases them; `"".join` glues them together.
- `cleaned[::-1]` is the string reversed. Equal to the original means palindrome.

Actual output: `True` , `True` , `False`

Practice: Check whether two words are anagrams.

Source of programs

Short beginner exercises in the style of public tutorials (W3Schools, Python.org tutorial, Real Python basics). Add the exact URL you used for each in your submission.